

SMARTROADMAP

Security Audit, Remediation & Feature Report

An end-to-end review of the SmartRoadmap monorepo (NestJS + Next.js), the vulnerabilities and defects found, everything that was fixed and built, the technology behind each feature, and how every item is verified.

Prepared for: Ahmed Khaled — Media Dose Agency

Date: 13 July 2026

Scope: apps/api · apps/web · packages/shared · infrastructure · CI

FINAL STATUS

70 automated tests passing (17 unit + 53 live end-to-end/security) · Both apps typecheck and build clean · Production Docker images ready.

Contents

1. Executive Summary

SmartRoadmap is a career-preparation and hiring platform: it generates an adaptive learning roadmap for a target role, verifies skills through quizzes, builds a CV, and matches candidates to jobs, with paid subscriptions and a recruiter portal.

The codebase was well structured but not safe to deploy. The review found five vulnerabilities that were exploitable by anyone with a browser — including a complete account takeover of every account in the system — plus four defects that would crash the server process on the first upstream failure. The frontend could not be deployed at all, and the README described a system that did not exist.

All of these have been fixed, and the platform was extended with the flows it was missing (password reset, email verification, refresh-token theft detection, real skill-gap analysis) and made deployable.

Headline findings

Severity	Issue	Impact
CRITICAL	POST /auth/google accepted a raw email address	Anyone could log in as ANY user, including admins
CRITICAL	No authorization guards anywhere in the API	Every endpoint was public: full user list, any CV, any profile
CRITICAL	userId was read from the request body	Edit anyone's profile; escalate yourself to admin
CRITICAL	Payment mock mode active in production	Free subscription upgrades without paying
CRITICAL	Infinite recursion in 4 error handlers	Process crash on the first OpenAI/PayPal failure
HIGH	Weak crypto (PBKDF2 1k rounds, hardcoded JWT secret)	Offline password cracking; token forgery
HIGH	Admin gate lived in localStorage	Two lines in DevTools made you an admin
HIGH	API URL hardcoded in 37 places	The frontend could not be deployed anywhere

Outcome

Area	Before	After
Authorization guards	0	Global deny-by-default + roles + ownership
Request validation	None	DTOs on every endpoint
Automated tests	1 (the Nest default)	70 (17 unit + 53 end-to-end/security)
Deployable	No	Yes — multi-stage Docker images
Crash-on-error paths	4	0
Largest file	1,808 lines	1,240 lines (logic extracted to hooks)

2. Vulnerabilities Found & Fixed

2.1 Account takeover via Google sign-in (Critical)

The endpoint trusted a client-supplied email address. There was no verification of any kind.

```
POST /auth/google { "email": "admin@smartroadmap.dev" } → valid token for that account
```

Fixed: the browser now obtains a Google-signed ID token (Google Identity Services) and the server verifies its signature and audience against Google's public keys using google-auth-library. A client-supplied email is never trusted. An account created with a password can no longer be taken over through Google (account-linking attack).

2.2 No authorization anywhere (Critical)

A search for @UseGuards across the whole API returned zero results. GET /auth/users returned every user in the system, with their emails, to anyone.

Fixed with three layers:

- JwtAuthGuard is registered globally (APP_GUARD). The API is deny-by-default: a route is only reachable without a token if it is explicitly marked @Public(). Forgetting a guard can no longer leak data.
- RolesGuard + @Roles('admin' | 'company') gate role-specific endpoints (user list, candidate pool, job posting).
- assertSelfOrAdmin() protects every /:userId and /:id resource, closing the IDOR class of bugs (CV, roadmap, quiz session, payment order).

2.3 Identity taken from the request body (Critical)

POST /auth/update-profile accepted { userId, data } — so any user could edit any other user's profile, and set their own role to admin.

Fixed: identity now comes only from the verified JWT via the @CurrentUser() decorator. The endpoint became PATCH /auth/me. The validation pipe strips unknown properties, so a spoofed userId or role never reaches the service. 'admin' cannot be self-assigned at registration, and role/email are not self-editable fields.

2.4 Free subscriptions (Critical)

Payment mock mode activated silently whenever the PayPal credentials were missing — including in production — and the capture endpoint had no authentication at all.

Fixed:

- The service throws at boot if PayPal credentials are missing under NODE_ENV=production. Mock payments are impossible in a live deployment.
- Prices are resolved server-side from the plan; the client never sends an amount.
- Capture verifies both that the order belongs to the caller and that the amount captured matches the plan price.
- A signed PayPal webhook (POST /payment/webhook, signature verified with PayPal) is now the source of truth for entitlements, and handles refunds and reversals by downgrading the user.

2.5 Weak authentication crypto (High)

Was	Now
Hand-rolled JWT with a hardcoded fallback secret committed to the repo	@nestjs/jwt; the server refuses to boot without a 32+ character JWT_SECRET
PBKDF2, 1,000 iterations	bcrypt, 12 rounds (configurable)

Was	Now
signature !== expected (timing attack)	bcrypt.compare (constant time) + a dummy compare on unknown emails
Password hash returned by queries	select: false on the schema
No refresh tokens	Access (15 min) + refresh (30 days) with rotation

2.6 Client-side admin gate (High)

The admin pages granted themselves a session: `localStorage.setItem('smart_token', 'demo-token')`. Two lines in DevTools made anyone an admin — and the backend was not checking anything either.

Fixed: the fake session is gone; admin pages verify identity against `GET /auth/me`, and the API re-checks the role on every request with `RolesGuard`. The UI gate is now convenience, not the security boundary.

2.7 Transport and abuse protections (High)

- CORS changed from origin: `'*'` (with credentials — not even a legal combination) to an allow-list driven by `FRONTEND_URL`.
- `helmet()` added for standard security headers.
- Global `ValidationPipe` with `whitelist: true` — unknown properties are stripped from every request body.
- Rate limiting via `@nestjs/throttler`: 100 req/min globally, 5 req/min on login/register (configurable with `AUTH_THROTTLE_LIMIT`).
- CV upload restricted to PDF, 5 MB maximum. An empty upload now returns 400 instead of silently returning fabricated data.

3. Stability Defects Fixed

3.1 Infinite recursion in four error handlers (Critical)

Four catch blocks called the function that had just failed:

```
catch (error) { return this.generateRoadmap(targetRole, skills); } // → recurses forever
```

Present in `LLMService.generateRoadmap`, `LLMService.generateQuiz`, `EmbeddingService.embed` and `PaymentService.createOrder`. On the first OpenAI or PayPal failure the process would recurse until the call stack blew — taking the API down.

Worse: the code called `require('openai')` but the `openai` package was not in `package.json` at all. The moment a real API key was configured, every AI call would throw `MODULE_NOT_FOUND`, hit the catch, and recurse.

Fixed: the mock responses were extracted into pure functions; fallbacks call those (a terminating path), never the public method again. The `openai` package was installed and a single shared client created. Payment failures now fail loudly with a 503 instead of silently degrading to a free mock order.

3.2 Silent MongoDB fallback to localhost (Critical)

A DNS failure resolving the Atlas SRV record caused the app to connect to a local MongoDB instead — meaning production traffic would be written to an empty local database with no error. Removed: `MONGODB_URI` is required and the app fails loudly.

3.3 Same-second token revocation bug (High)

Found while building the refresh-rotation tests. A JWT's `iat` claim is in whole seconds, but `tokensValidFrom` is stored with millisecond precision — so a token minted in the same second as the account was created looked "older" than the revocation threshold and was rejected. A one-second tolerance now prevents valid tokens from being revoked.

3.4 Work done on the login hot path

Every single login re-seeded demo notifications and support messages (including a fabricated "97% match at Stripe" notification). This is now an onboarding service that runs once at signup, is fire-and-forget, and is disabled in production. Job seeding was moved out of a constructor (which was firing un-awaited database writes) into the proper lifecycle hook.

3.5 Documentation that described a different system

The README advertised LangChain, Qdrant, BullMQ, Redis queues, React Flow and an Adzuna scraper. None of them were in the dependency tree. Qdrant has since been implemented for real; the rest were removed from the README, which now states honestly what exists and what does not.

4. Features & the Technology Behind Each

4.1 Technology stack

Layer	Technology	Why
Backend	NestJS 11	Modular DI, guards, pipes, lifecycle hooks
Database	MongoDB + Mongoose	Flexible documents for CVs, roadmaps, sessions
Vector search	Qdrant	Semantic retrieval layer — built, index not yet populated
AI	OpenAI (gpt-4o, gpt-4o-mini, text-embedding-3-small)	Roadmaps, quizzes, CV parsing, embeddings
Auth	@nestjs/jwt · bcrypt · google-auth-library	Access/refresh JWTs, hashed passwords, verified Google
Validation	class-validator + Zod	Per-endpoint DTOs; fail-fast config validation
Hardening	helmet + @nestjs/throttler	Headers, brute-force protection
Email	Resend (console transport in dev)	Password resets, email verification
Frontend	Next.js 14 (App Router) + React 18	File-based routing
Styling / i18n	Tailwind + DaisyUI · custom AppContext	Theming; English (LTR) + Arabic (RTL)
Payments	PayPal REST + signed webhooks	Subscriptions
Media	Cloudinary	Avatar uploads
Docs	@nestjs/swagger	Auto-generated /docs
Infra	Docker Compose · multi-stage Dockerfiles	Dev stack and production images
CI	GitHub Actions	lint → build → unit tests → live smoke suite

4.2 Adaptive learning roadmap (core)

Generates a dependency-ordered set of learning modules for a target role, then reshapes itself based on quiz performance.

- Generation: OpenAI gpt-4o returns a structured JSON roadmap (modules, prerequisites, hours, difficulty). A deterministic mock is used offline.
- Pass: completing a module's quiz auto-unlocks every module whose prerequisites are now satisfied.
- Fail (new): a shorter remedial module is generated from the exact questions the learner missed, so nobody gets stuck. The failed module is marked failed rather than left in limbo.

4.3 Adaptive skill assessment

- Questions generated by OpenAI gpt-4o-mini in structured JSON.
- Adaptive difficulty: two correct answers in a row raise it, two wrong lower it.
- Weighted scoring (easy 1 · medium 1.5 · hard 2), 70% pass mark.
- A quiz session can only be answered by its owner.

4.4 CV builder & AI parser

- pdf-parse extracts the text; OpenAI converts it into structured JSON (personal, experience, education, skills, projects). A regex heuristic parser is the fallback.
- AI rewrites résumé bullets into active, results-driven phrasing.
- Uploads restricted to PDF, 5 MB, enforced server-side.
- Code health: the 1,808-line page was split into a presentational component, a useCvEditor hook, and icon/type modules.

4.5 Job matching & skill-gap analysis

- Scoring today is a deterministic skill-overlap score: verified skills (from completed roadmap modules) against the job's required skills. Exact and fully explainable. Every match returns a matchScore and a skillsGap.
- Semantic search is built but not yet wired: RAGService implements the full Qdrant path (collections created on boot, embeddings via text-embedding-3-small, cosine retrieval, batched upsert) but the matching service does not call it yet and the index is empty. The next step is a hybrid score plus a job importer.
- New: POST /hiring/jobs/:jobId/close-gap recomputes the gap server-side against the learner's verified skills and writes the missing ones into their active roadmap as new modules. "You are a 78% match, you are missing Docker" now turns into an actual next step. (The button for this previously ran a setTimeout and only repainted the screen.)

4.6 Account recovery & session security (new)

- Forgot password: a hashed, single-use, one-hour token is emailed. The endpoint always returns the same 200 whether or not the address exists, so it cannot be used to enumerate users.
- Reset password: verifies the hashed token, re-hashes with bcrypt, and revokes every session on every device.
- Email verification: a 24-hour token is sent at signup; resend is throttled to 3 per 5 minutes.
- Email delivery: console in development, Resend in production — and RESEND_API_KEY is required in production, so a reset email can never silently vanish.
- Refresh-token rotation with theft detection: every refresh burns the old token. Tokens are stored hashed (SHA-256), one per device. If an already-spent token is presented it means it was replayed — the API revokes every session for that account and logs it.

4.7 Payments & subscriptions

- PayPal orders with server-side pricing and amount verification on capture.

- Signed webhook as the source of truth for entitlements; refunds and reversals downgrade the user automatically.
- Subscription state (plan, status, expiry) lives on the user record. Previously, "upgrading" only changed the user's UI theme.

4.8 Platform & operations

- Zod config validation: the API refuses to boot on an invalid or incomplete configuration. `MOCK_MODE` is an explicit opt-in and is rejected in production.
- Health probe at `GET /health` (checks the MongoDB connection); Swagger at `/docs` outside production.
- Global exception filter with a per-request `requestId` and structured logging.
- Central API client on the frontend (`src/lib/api.ts`): base URL from `NEXT_PUBLIC_API_URL`, automatic bearer token, transparent refresh on 401.

5. Security Model (Reference)

5.1 Token storage

Token	Stored where	Lifetime	Rationale
Refresh	httpOnly cookie, Path=/auth, Secure + SameSite=None in production	30 days	JavaScript cannot read it, so XSS cannot steal the session. It is not sent on any other route, so the CSRF surface is nil.
Access	Memory only (dies with the tab)	15 minutes	Silently re-minted from the cookie on reload.
smart_user	localStorage	—	A UI cache only. Never trusted for authorization; the server re-checks the JWT and role on every request.

5.2 Authorization layers

- Layer 1 — Authentication: global JwtAuthGuard, deny-by-default.
- Layer 2 — Role: RolesGuard + @Roles(). Admin and company endpoints.
- Layer 3 — Ownership: assertSelfOrAdmin() on every per-resource route.
- Layer 4 — Input: ValidationPipe with whitelist, so spoofed fields never reach a service.

6. Verification & Testing

Nothing in this report is asserted without a test behind it. Every fix and every feature is covered by an automated check, a manual QA step, or both.

6.1 Automated suites

Suite	Command	Count	What it proves
Unit tests	npm test	17	Auth service (bcrypt, no admin self-assign, Google-on-local rejection, hashed refresh tokens, no user enumeration), ownership utility, cookie flags, health endpoint
Live end-to-end / security	npm run smoke	53	Runs against a RUNNING server: proves the deployed system behaves correctly, not a mock

Latest run: 53 passed, 0 failed (verified against a live API with an in-memory MongoDB).

6.2 What the smoke suite checks

Section	Checks	Examples
0 · Health	1	API and MongoDB are up
1 · Auth	7	Weak passwords rejected; refresh token is httpOnly and absent from the body; unknown emails return an identical error (no enumeration)
1b · Rotation	2	Refreshing rotates the token; replaying a spent token revokes ALL sessions (theft detection)
1c · Recovery	5	Forgot-password returns an identical 200 for known and unknown emails; forged reset/verification tokens rejected
2 · Deny-by-default	9	Seven protected routes return 401 without a token; forged JWTs rejected
3 · Escalation	4	A learner cannot list users, self-assign admin, escalate via profile update, or post a job
4 · Google	2	A raw email is rejected; a fabricated ID token is rejected
5 · IDOR	4	User A cannot read user B's CV, roadmap or job matches
6 · Roadmap	7	Generation, ownership, validation, adaptive quiz, and that the roadmap reacts to the outcome
6b · Gap analysis	2	Matches expose a gap; closing it really adds modules to the roadmap
7 · Payments	5	Server-side pricing; another user cannot capture your order; unsigned webhooks rejected
8 · Misc	4	Empty CV upload rejected; newsletter endpoint exists; login is rate limited

6.3 Manual QA

QA-CHECKLIST.md walks the entire site in a browser (sections A-I): authentication and cookies, the admin gate (including an attempt to fake an admin session in DevTools), the adaptive roadmap pass and fail paths, the CV builder, hiring and gap-closing, payments, messaging, Arabic/RTL, and resilience (API down, token expiry, token theft).

6.4 Continuous integration

lint → typecheck + build → unit tests → boot the API → 53-check smoke suite

Defined in `.github/workflows/ci.yml`, running on every push and pull request with a MongoDB service container.

7. Deployment

7.1 Running it

```
cp .env.example apps/api/.env          # fill in the secrets
cp apps/web/.env.local.example apps/web/.env.local
docker compose up -d                   # MongoDB + Redis + Qdrant
npm install && npm run dev              # API :3000 · Web :3001 · Docs :3000/docs
```

7.2 Production

```
docker compose -f docker-compose.prod.yml up --build
```

Multi-stage Dockerfiles for both apps: dependencies pruned, non-root users, a healthcheck on the API, and Next.js standalone output for a minimal web image.

7.3 Required in production

The API will not start unless these are present, and `MOCK_MODE=true` is rejected outright:

Variable	Purpose
MONGODB_URI	Database (no localhost fallback any more)
JWT_SECRET / JWT_REFRESH_SECRET	Minimum 32 characters each; no fallback secret exists in code
GOOGLE_CLIENT_ID	Audience for verifying Google ID tokens
OPENAI_API_KEY	AI generation
PAYPAL_CLIENT_ID / SECRET / WEBHOOK_ID	Payments and webhook signature verification
RESEND_API_KEY	Password-reset and verification emails

8. Recommended Next Steps

Ordered by value against effort. The first two are the product's differentiators and are not yet built.

Tier 1 — Differentiators

Feature	Why it matters	Effort
Verifiable Skill Passport	Each passed quiz becomes a signed credential with a public URL a recruiter can check. Skills on LinkedIn are claims; yours would be verified. This is the defensible moat.	2–3 weeks
Wire up semantic matching	RAGService (Qdrant) is implemented and tested but the matching service does not call it, and the index is empty. Add a job importer and blend cosine similarity with the existing overlap score.	1 week
Recruiter dashboard	Vector search over candidates, a hiring pipeline, ATS integrations. This is the side of the marketplace that pays.	2 weeks

Tier 2 — Growth & retention

- AI mock interview with a scorecard — the feature people actually pay for individually.
- ATS score for the CV against a specific job description, with missing-keyword detection.
- Streaks, XP and weekly digests (the Redis service in the compose file is currently unused; BullMQ would drive the emails).
- Arabic-first content. The UI already supports RTL; the roadmaps and resources do not. Every major competitor is English-first — this is a real edge in the MENA market.

Tier 3 — Hardening & operations

- Two-factor authentication (TOTP) for admin and recruiter accounts.
- Audit log: who viewed which CV, who changed whose role.
- Sentry integration (the requestId plumbing already exists).
- Playwright end-to-end tests for the payment and auth journeys in a real browser.
- Split the remaining large pages (cv 1,240 · company 619 · pricing 569 lines) into presentational components.

9. Deliverables

File	Contents
SmartRoadmap-FINAL.zip	The complete, fixed and extended monorepo
FEATURES-REPORT.md	Every feature and the technology behind it (English, in-repo)
QA-CHECKLIST.md	Manual browser walkthrough, sections A-I
scripts/smoke-test.mjs	The 53-check live security and feature suite
AI-ARCHITECTURE.md	Where the AI lives in the code, the five touchpoints, model choices, fallbacks
scripts/README.md	How to run the suite, including with an in-memory MongoDB
README.md	Rewritten: an honest description of what exists
.env.example	Every variable, with production requirements marked
docker-compose.prod.yml + Dockerfiles	Production deployment
.github/workflows/ci.yml	lint → build → unit tests → live smoke suite

Verification summary: 17 unit tests and 53 live end-to-end/security checks pass; both applications typecheck and build cleanly; production images build and run.